M2 LID – NoSQL and NewSQL

Cypher

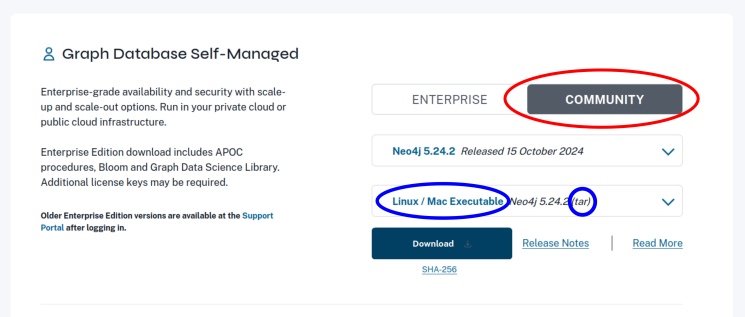
2026 – 2027

▶ **Exercise 1** – *Download and configure Neo4j and the dataset (to do only once)*

1. Download the files `neo4j-community-5.26.0-unix.tar.gz` and `neo4j.dump` from elearning.

   Alternatively, the newest tarball can may be downloaded from `https://neo4j.com/deployment-center/#gdb-tab` (untested since 2024)

   — Scroll down to section *Graph Database Self-Managed*.



   — Select the community edition (circled in red) and the tar.gz archive for Linux (circled in blue).

2. Extract the tarball in somewhere. Choose a directory without special characters in the path (in particular, do not use any directory under `Téléchargements`).

   Paths are given relative to that directory in the following.

3. In file `conf/neo4j.conf`, uncomment the following line by removing the `#`.

   `#dbms.security.auth_enabled=false`

4. Create the subdirectory `data/dumps` if it does not exists.

5. Download the file `neo4j.dump` from elearning and move it in subdirectory `data/dumps`.

6. Launch a terminal in the neo4j directory.

7. Execute the following command in the terminal to import the data in neo4j.

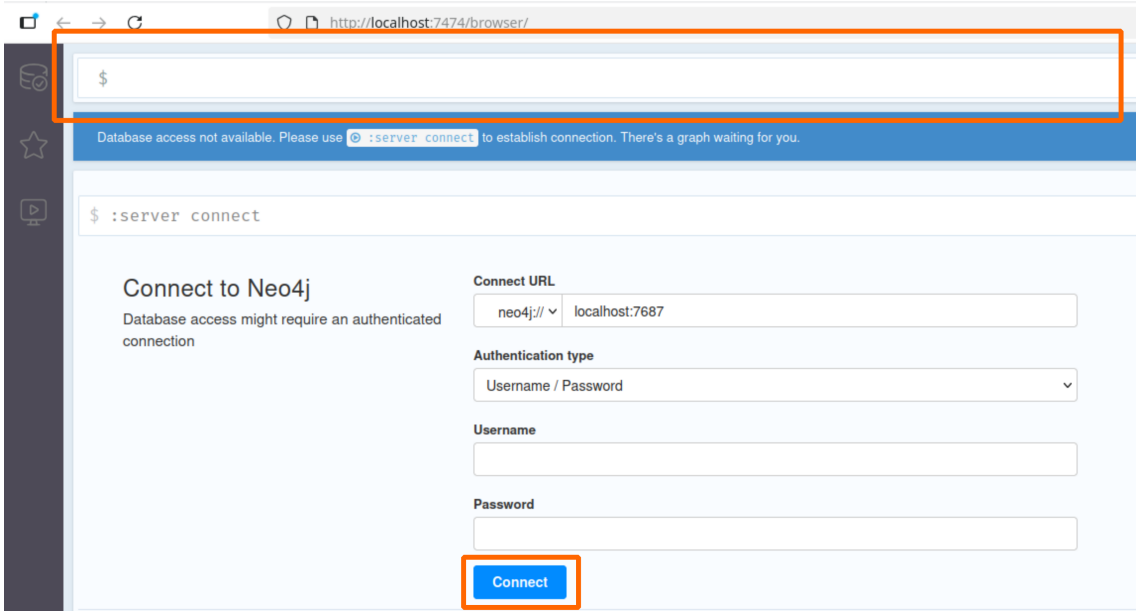
   `bin/neo4j-admin database load neo4j --overwrite-destination=true --verbose`

▶ **Exercise 2** – *In order to launch Neo4j in a web browser, do the following.*

1. Execute the following command and leave the terminal open.

   `bin/neo4j console`

2. In a web browser go to `http://localhost:7474`. It should show something like the figure below. You might have to clic on the blue "connect" button at the bottom of the page.



3. Cypher queries are written in the zone framed in orange above, and launched with `CTRL+ENTER`. As a test, launch the query `MATCH (a) RETURN count(a)` which should return 69165 if the dataset was correctly imported.



▶ **Exercise 3** – *Explore the nodes*

1. Consider the clause "`MATCH (n:OSMNode)`". Describe what it does : What is looking for ? Is there a label ? Is there a property ? Is there a variable ?
2. The clause above is **not** a query by itself. Modify it in order to be able to run it in Neo4j. We expect 29640 results.
3. Using `LIMIT`, make the previous query return only 50 nodes (chosen arbitrarily by the DBMS).
4. Return the values associated with the key `name` among nodes labeled by `OSMNode`. We expect to obtain 216 values, none of which is `NULL`.
5. Make the previous query return only the 50 `name` that are the smallest according to the default order on strings.
6. Return the `type` of the `PointOfInterest` whose `name` is `"Delacorte Musical Clock"`.
7. How many `PointOfInterest`'s is there with that `type` ?
8. Using two `MATCH`'s, find all `PointOfInterest`'s within 200 distance[1] unit of the Delacorte Musical Clock. Three answers are expected.

▶ **Exercise 4** – *Query plan and indexes*

1. Compare the query plans[2] of the following query.
   — Look for the Delacorte Musical Clock by name, **without** using any label.
   — Look for the Delacorte Musical Clock by name, using `PointOfInterest`.
2. In order to improve performance, we want to add an index[3].
   — Display existing indexes.
   — Add an index named *index_poi_name* on on key `name` for nodes with label `PointOfInterest`.
   — Display existing indexes to check if it appears.
3. Compare the new query plan of the two queries from question 1.

▶ **Exercise 5** – *Explore the relations*

1. Use the query `MATCH ()-[:OSMEdge]->()` that looks for relations of type `OSMEdge`.
2. Now that you have noted that the previous query returns no result, browse the data to find the relation type we need to navigate in the graph.
   Do you think that the direction of this type is meaningful ?
3. Compute all relations that are adjacent to the Delacorte Musical Clock. If the computed relations are not displayed graphically, find out why and fix it.
4. Display the neighborhood of the Delacorte Musical Clock at distance 5.
5. Find one path (using only relation with types from question 2) from the node with `name` equal to `"Delacorte Musical Clock"` to the one named `"Balto"`

1. https://neo4j.com/docs/cypher-manual/5/functions/spatial/#functions-distance
2. https://neo4j.com/docs/cypher-manual/5/planning-and-tuning/
3. https://neo4j.com/docs/cypher-manual/5/indexes-for-search-performance/

▶ **Exercise 6** – *Aggregation*

1. Count the number of `OSMNode` nodes with a non-null `name`.
2. Compute each `Intersection` node and the number of adjacent `ROUTE` it has.
3. Compute the *no-choice intersections*, that is the `Intersection`'s that have exactly two adjacent `ROUTE`'s. We are expecting 146 answers.
4. Compute each `OSMNode` node that is not a `PointOfInterest` and that is a dead-end (adjacent to only one `ROUTE`). We are expecting 150 answers.
5. Using `reduce`, compute the number of relations in the path from the Delacorte Musical Clock to Balto (see question 5 of exercice 5).
6. Same question, but with the shortest path in `length`.
7. Using `reduce`, find the shortest path(s) in number or relations from the Delacorte to Conservatory Water.
8. You should have observed that the previous query did not terminate. Why ?

▶ **Exercise 7** – *Modifying the graph (soft)*

1. For each `OSMNode` node, add the new property `deadend` to `false`. Then, set the property `deadend` to `true` for nodes that are deadends, according to question 4 of exercice 6.
2. Verify that the Azalea Pond is not marked as a deadend, but that the node with `node_osm_id` equal to `42432401` is marked as a deadend.
3. Add a new property `mergeable` that is set to `true` for `ROUTE` relations that are adjacent to a a no-choice intersection, according to question 3 of exercice 6). All other `ROUTE` relations should have this property set to `false`.

▶ **Exercise 8** – *Modifying the graph (hard)*

In this exercice, we will modify the graph, and executing wrong queries might delete part of it in an unrecoverable way. Be ready to terminate your database and recreate it from scratch.

1. Delete each `OSMNode` node that is a deadend (see question 1 of exercice 7).
2. Verify that the Azalea Pond is still in the graph, but that the node with `node_osm_id` equal to `42432401` has been deleted.
3. Merge each maximal path of `mergeable` relations into a single new relation (see question 3 of exercice 7). The old relations should be deleted. The new relation